

UNIVERSIDADE FEDERAL DO RIO GRANDE DO NORTE (UFRN)

CIÊNCIA & TECNOLOGIA

Documento de Descrição do Projeto Final
Etapas de Definição do Escopo e Viabilidade

Mercury Racing

*"A sua pole position para dados, estatísticas e cronogramas da Fórmula
1 em tempo real."*

FRANKLIN SOARES
MATRÍCULA: 20250051696
TURMA: 2026.1

NATAL / RN
2026

1. Descrição Geral

Resumo

O **Mercury Racing** é uma aplicação web de página única (SPA) desenvolvida em React que permite a fãs de automobilismo e entusiastas de *sim racing* consultar dados, estatísticas e cronogramas da Fórmula 1 em tempo real, de forma rápida, intuitiva e com alta performance computacional.

Problema

Hoje, o público que acompanha o esporte a motor enfrenta dificuldade em encontrar dados estatísticos centralizados e de rápido acesso. Isso ocorre porque a maioria dos portais esportivos foca primariamente na veiculação de notícias, apresentando interfaces pesadas, poluídas e com carregamento lento para consultas simples, como verificar a pontuação atualizada do campeonato ou conferir os horários das sessões convertidos para o fuso horário local.

Proposta & Não-objetivos

O sistema oferece uma visualização limpa e focada em dados puros, entregando tabelas interativas de classificação (Pilotos e Construtores) e o calendário detalhado de corridas consumidos diretamente da API pública *OpenF1*. O grande foco é proporcionar uma experiência de usuário responsiva, moderna e sem tempo de espera (recarregamentos) desnecessário.

Como restrição de escopo estrutural, **não pretendemos** cobrir outras categorias do automobilismo mundial, nem construir um sistema de autenticação ou banco de dados relacional próprio (visto que o desenvolvimento de um *backend* próprio está expressamente fora do escopo desta etapa do projeto). A aplicação focará estritamente no processamento e na interface no lado do cliente (Front-End).

2. Público-Alvo & Personas

Público-Alvo Geral

O público-alvo da aplicação é composto por fãs assíduos de Fórmula 1, entusiastas de esportes a motor e praticantes de *sim racing* (automobilismo virtual) que buscam uma fonte rápida, direta e confiável para consultar dados do campeonato e cronogramas de corridas, além de ferramentas analíticas profundas.

Persona: Lucas, o Entusiasta Analítico

- **Perfil:** Lucas tem 24 anos, é estudante de tecnologia e piloto amador de *sim racing*. Ele é fascinado por dados de telemetria, estratégias de corrida e acompanha rigorosamente todas as sessões do fim de semana de Fórmula 1.
- **Contexto:** Durante os finais de semana de GP, Lucas assiste às corridas em uma tela enquanto conversa com seus amigos no Discord e acompanha estatísticas em um segundo monitor. Ele gosta de analisar a fundo a consistência dos pilotos, cruzar dados reais da F1 e comparar desempenhos lado a lado para embasar suas análises e discussões sobre o esporte.
- **Dor Principal:** Lucas se frustra com sites tradicionais que demoram para carregar devido a anúncios e não oferecem profundidade técnica. Ele perde muito tempo calculando fusos horários manualmente e não possui uma plataforma unificada onde possa criar uma conta para salvar suas preferências e acessar comparativos diretos (*Head-to-Head*) e análises avançadas de telemetria de forma rápida durante o calor da corrida.

3. Listagem de Funcionalidades

As funcionalidades abaixo foram mapeadas utilizando a estrutura de *User Stories*, categorizadas em essenciais (MUST) e desejáveis (NICE). Os atores foram definidos entre **Visitante** (área pública) e **Usuário Autenticado** (área protegida do painel analítico).

Obrigatórias (MUST)

MUST-01 // Autenticação e Acesso ao Painel

Como visitante, eu quero criar uma conta com e-mail e senha para acessar as ferramentas analíticas avançadas.

- *Critérios de aceitação:*
 - Validação de formato de e-mail e força da senha no cliente.
 - Integração com o Firebase Auth para criação e login, redirecionando para a rota protegida do painel após o sucesso.

MUST-02 // Classificação Geral (Pilotos e Construtores)

Como visitante, eu quero visualizar as tabelas de classificação atualizadas para saber quem lidera os campeonatos mundiais.

- *Critérios de aceitação:*
 - Consumir a rota de `standings` da API OpenF1.

- Permitir alternar entre a tabela de Pilotos e a de Construtores através de abas ou botões visíveis na interface.

MUST-03 // Calendário de Corridas

Como visitante, eu quero ver o cronograma completo da temporada para me programar para os finais de semana de GP.

- *Critérios de aceitação:*
 - Exibir uma lista ou grade com todos os Grandes Prêmios do ano (circuito e datas).
 - Destacar visualmente qual é a próxima corrida programada no calendário.

MUST-04 // Perfil Dinâmico do Piloto

Como visitante, eu quero clicar no nome de um piloto na tabela para ver seus detalhes individuais.

- *Critérios de aceitação:*
 - O clique deve redirecionar para uma rota dinâmica (ex: `/drivers/:id`).
 - A página deve exibir informações específicas, como número do carro e nacionalidade.

MUST-05 // Comparativo Head-to-Head

Como usuário autenticado, eu quero selecionar dois pilotos e comparar seus desempenhos lado a lado para debater consistência técnica.

- *Critérios de aceitação:*
 - Acessível apenas na área logada.
 - Renderizar componentes visuais (barras ou métricas textuais) comparando pontos totais e vitórias entre os dois pilotos selecionados.

MUST-06 // Telemetria ao Vivo (Real-Time)

Como usuário autenticado, eu quero acessar o painel de telemetria em tempo real (*Live Timing*) para acompanhar posições e tempos de volta durante as corridas.

- *Critérios de aceitação:*
 - Acessível apenas na área logada (`/app/realtime`).
 - Exibir uma tabela lateral com tempos de volta atualizados e um espaço dedicado ao mapa do circuito.

Desejáveis (NICE)

NICE-01 // Conversão de Fuso Horário

Como visitante, eu quero que os horários das sessões sejam exibidos automaticamente no meu fuso horário local.

NICE-02 // Tema Escuro (Dark Mode)

Como usuário da aplicação, eu quero alternar a interface para um tema escuro para maior conforto visual.

4. Mapa do Site (Sitemap)

Abaixo está a estrutura de roteamento da aplicação (organizada com *React Router*), mapeando as páginas abertas para visitantes e as rotas protegidas exclusivas para usuários autenticados.

```
/                # Landing Page / Home - Pública
|-- /standings   # Classificação Geral (MUST-02) - Pública
|-- /calendar    # Calendário de corridas (MUST-03) - Pública
|-- /drivers/:id  # Perfil do Piloto (Dinâmica / MUST-04) - Pública
|-- /login        # Tela de Autenticação (MUST-01) - Pública
|-- /app          # [Auth] Painei Analítico - Protegida (Requer Login)
|   |-- /app/h2h   # Comparativo Head-to-Head (MUST-05) - Protegida
|   \-- /app/realtime # Telemetria ao Vivo (MUST-06) - Protegida
\-- /404           # Página de Erro / Not Found - Pública
```

Checklist de Requisitos do Mapa

- **Mínimo de 5 rotas distintas:** O diagrama apresenta 8 rotas mapeadas.
- **Rota dinâmica:** Implementada na visualização de perfil individual (`/drivers/:id`).
- **Página de erro:** Rota de *fallback* implementada em `/404`.
- **Indicação de proteção:** Rotas aninhadas em `/app` exigem validação de sessão (*Firebase Auth*).
- **Coerência com Funcionalidades:** Todas as 6 funcionalidades obrigatórias (MUST) possuem rotas dedicadas e correspondentes.

5. Wireframes

Os wireframes abaixo foram desenvolvidos em baixa fidelidade para mapear a disposição dos blocos, a hierarquia da informação e a navegação da aplicação.

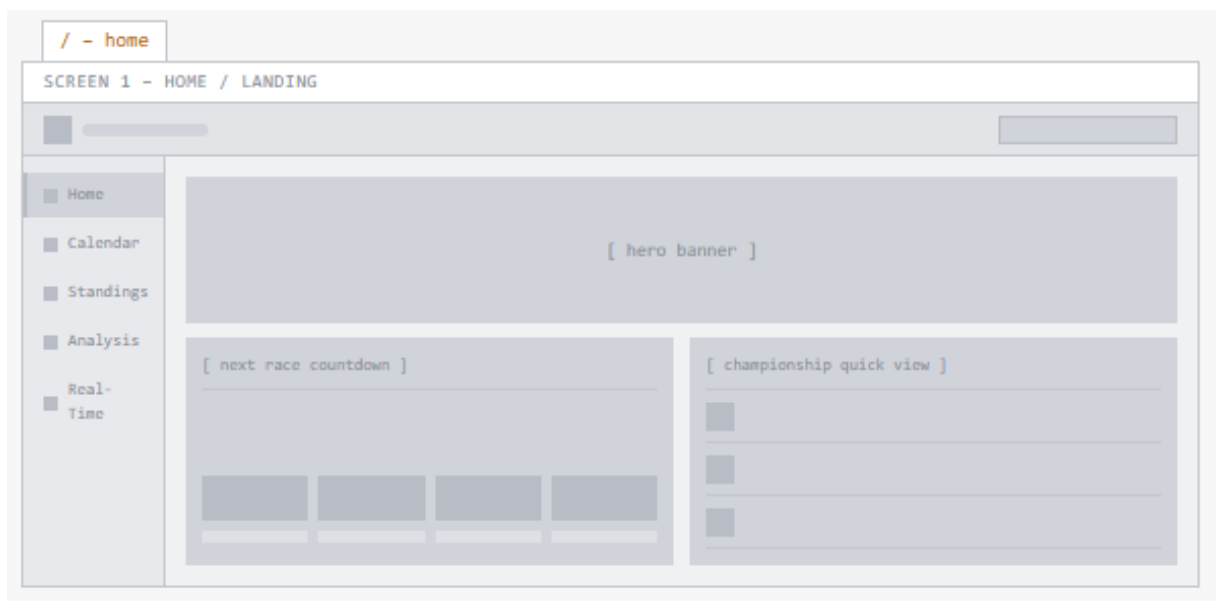
Layout Global (Presente em todas as telas)

A aplicação utiliza um padrão de *Dashboard* clássico para acomodar as ferramentas de análise avançada:

- **Cabeçalho (Header):** Fixo no topo, contendo a logo à esquerda e o botão de "Login / Dashboard" à direita.
- **Menu Lateral (Sidebar):** Fixo à esquerda, contendo a navegação principal (Home, Calendar, Standings, Analysis, Real-Time).

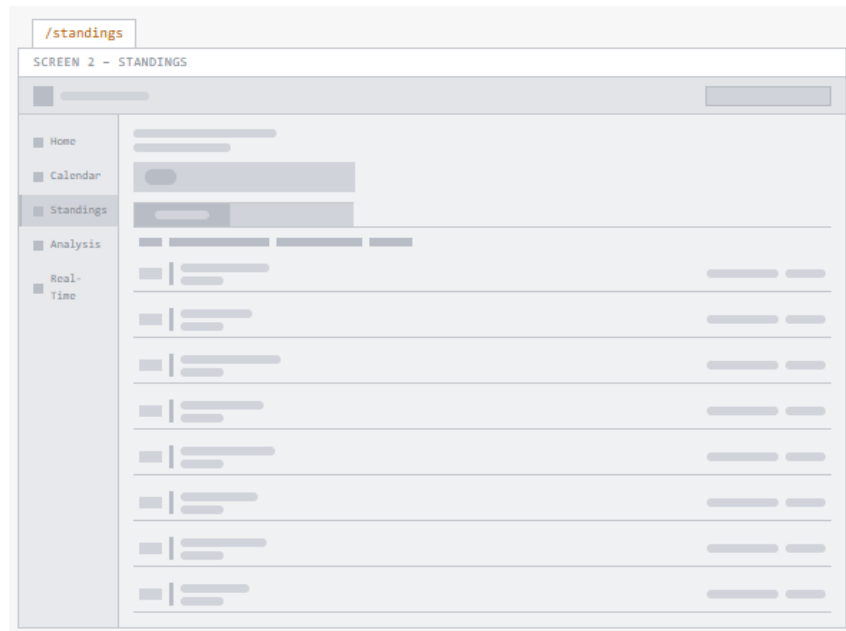
Tela 1: Landing Page / Home (/)

Apresenta um *Hero Banner* superior de destaque. Abaixo, a tela é dividida em dois widgets principais: um contador para a próxima corrida (*Next Race Countdown*) e uma lista rápida com o Top 3 do campeonato (*Championship Quick View*).



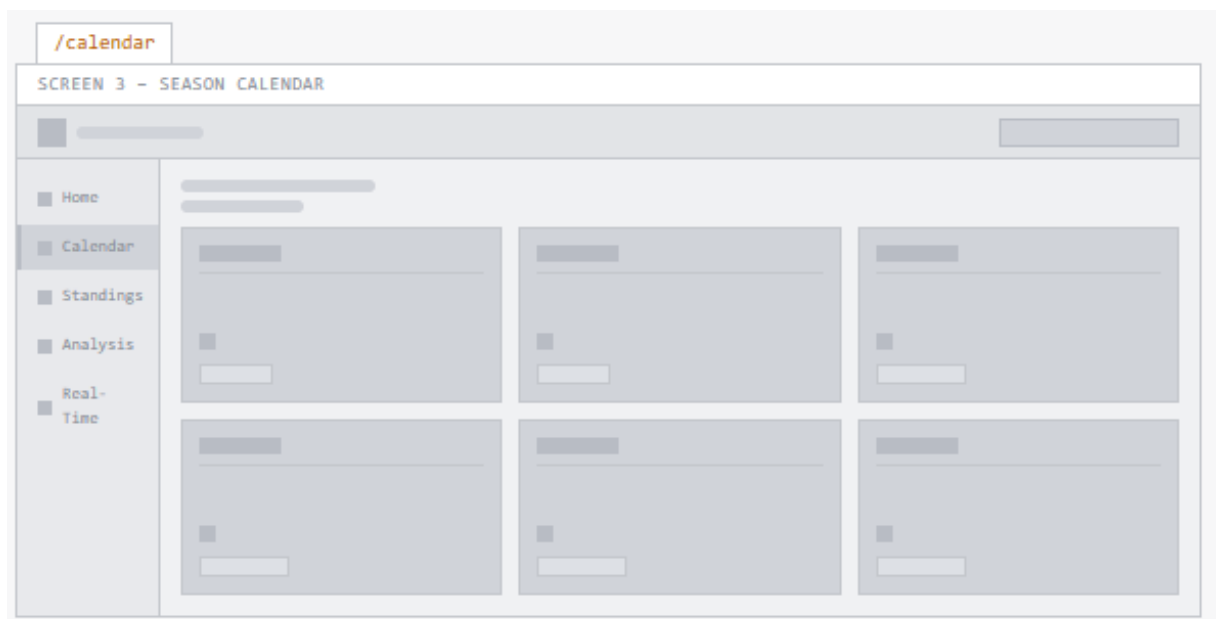
Tela 2: Classificação Geral (/standings)

Exibe o título da seção com duas abas largas para alternar a visão entre Pilotos e Construtores. Logo abaixo das abas, apresenta a tabela de dados estruturada ocupando a maior parte da área útil para leitura rápida das posições.



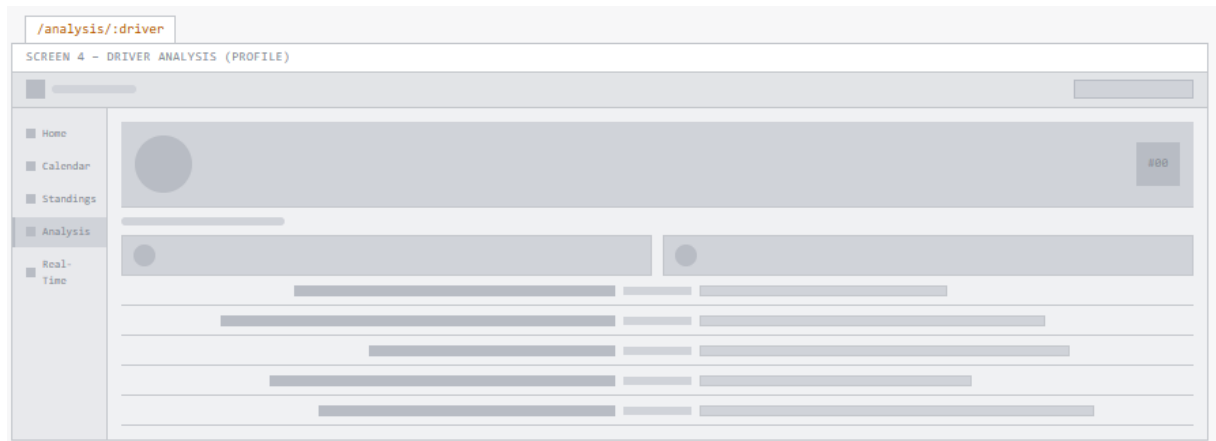
Tela 3: Calendário da Temporada (/calendar)

Apresenta as etapas do campeonato organizadas em um formato de grade (*Grid*), onde cada corrida é representada por um *card* contendo linhas para circuito, data e horário.



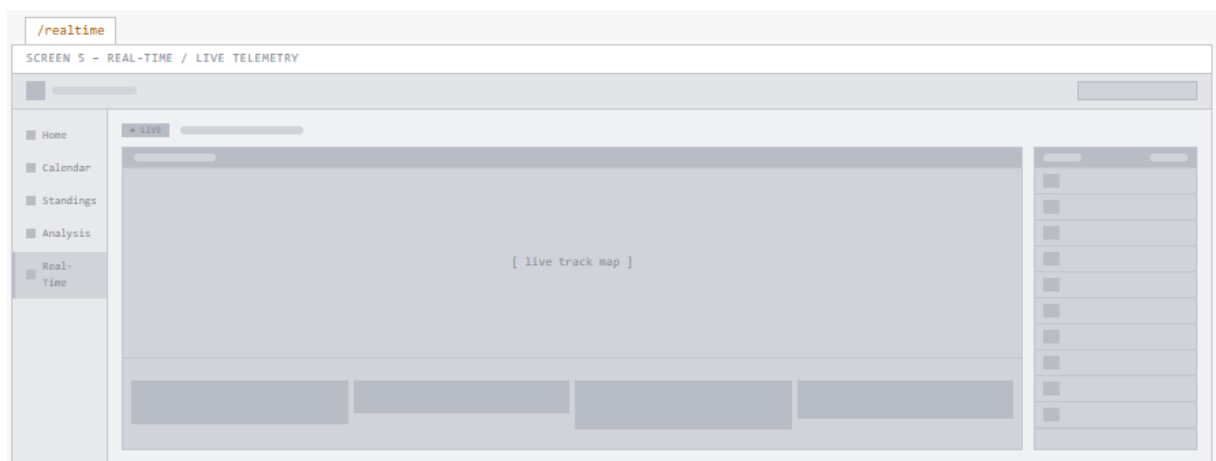
Tela 4: Análise de Piloto (/analysis/:driver)

A *Hero Section* exibe um espaço circular para a foto do piloto e blocos de texto para número/equipe. A parte inferior é dedicada à ferramenta de comparativo (*Head-to-Head*), com colunas lado a lado e gráficos de barra simples para comparar métricas.



Tela 5: Telemetria ao Vivo (/realtime)

Tela focada em dados instantâneos. O centro da tela é dominado por um espaço reservado para o mapa do circuito (*Live Track Map*), acompanhado de uma tabela lateral fina para exibir o painel de tempos e posições atualizadas (*Live Timing Tower*).



6. Stack Técnica

A arquitetura da aplicação foi desenhada respeitando as restrições do projeto (sem *backend* próprio) e utilizando tecnologias modernas focadas em performance e componentização no *client-side*.

Tecnologias Obrigatórias

- **React 18+:** Biblioteca principal para a construção da *Single Page Application* (SPA) baseada em componentes funcionais.
- **React Router (v6):** Gerenciamento de rotas e navegação da aplicação, isolando as áreas públicas das áreas protegidas (/app).
- **GitHub Público:** O controle de versão e a hospedagem do código-fonte serão mantidos de forma aberta em um repositório no GitHub.

Decisões de Livre Escolha (Justificadas)

- **Build Tool: Vite.** Escolhido no lugar do *Create React App* (CRA) por oferecer um *Hot Module Replacement* (HMR) significativamente mais rápido e um *build* otimizado.
- **Linguagem: TypeScript.** Essencial para tipar rigorosamente as respostas da API da F1 (pilotos, construtores, cronogramas), prevenindo erros de *runtime*.
- **Estilização: Sass (SCSS).** Permite criar uma arquitetura CSS modular e reutilizável (utilizando variáveis para cores como Vermelho Corrida e Carbono), ideal para manter a consistência visual em componentes independentes.
- **Gerenciamento de Estado UI: Zustand.** Será utilizado para gerenciar estados globais simples, como a alternância do Tema Escuro (*Dark Mode*), por ser menos verboso que o Redux e mais performático que a *Context API* para atualizações frequentes.
- **Consumo de API: Axios.** Facilita a configuração de *interceptors* e o tratamento de erros globais ao realizar *fetch* nos dados da *OpenF1*.
- **Ícones: Phosphor Icons.** Biblioteca de ícones consistente e moderna para compor a interface.

Infraestrutura Adicional

- **Autenticação: Firebase Authentication.** Como a criação de um *backend* próprio está fora de escopo, o Firebase será responsável por gerenciar o fluxo de *login* seguro, emitindo o *token* necessário para liberar o acesso ao painel analítico.

7. Fontes de Dados

Para atender à restrição arquitetural de não desenvolver um *backend* próprio, a aplicação consumirá informações inteiramente por meio de APIs públicas, serviços de *Backend as a Service* (BaaS) e arquivos estáticos estruturados localmente.

1. API Pública: OpenF1

A fonte primária de dados esportivos e telemetria será a API pública e gratuita da **OpenF1**. Ela fornecerá todos os dados oficiais da categoria de forma dinâmica:

- **Endpoints de Campeonato e Calendário:** Consumidos para renderizar as tabelas de classificação (Pilotos e Construtores) e a grade de corridas da temporada atual.
- **Endpoints de Pilotos:** Utilizados para popular a página dinâmica de análise (`/drivers/:id`) com informações descritivas, como número do carro, sigla, país e equipe.
- **Endpoints de Telemetria e Live Timing:** Consumidos na área logada (`/app/realtime`) para construir os gráficos comparativos de *Head-to-Head* e extrair tempos de volta.

2. Firebase

O **Firebase** será utilizado para gerenciar a sessão e as permissões de acesso da plataforma:

- **Firebase Authentication:** Responsável por gerenciar o fluxo de criação de conta e *login*, garantindo a emissão de *tokens* seguros para liberar o acesso ao painel analítico (`/app`).

3. Mocks e JSON Locais

Como APIs esportivas puras retornam apenas dados brutos e raramente fornecem ativos visuais de interface, o projeto utilizará arquivos estáticos locais (em formato `.json`) para complementar as respostas da API:

- **Mapeamento Visual (`themeData.json`):** Um dicionário de dados que associa o ID numérico de cada equipe retornada pela API à sua respectiva cor hexadecimal oficial (ex: Ferrari = `#E80020`, McLaren = `#FF8000`), garantindo fidelidade visual nas tabelas e gráficos.
- **Dados de Fallback (Mocks):** Estruturas de respostas *fakes* armazenadas localmente para permitir que a interface e as ferramentas de simulação continuem sendo desenvolvidas e testadas mesmo caso a API pública sofra instabilidades ou limite de requisições (*rate limit*).

8. Riscos e Atenções

O desenvolvimento de aplicações *client-side* dependentes de serviços de terceiros e dados rápidos envolve desafios técnicos. Abaixo estão mapeados os principais riscos do projeto e seus respectivos planos de mitigação.

- **Risco 1: Instabilidade e Limites da API Pública (OpenF1)**

Problema: Por ser uma API gratuita, a *OpenF1* pode sofrer quedas de servidor em dias de corrida intensa ou aplicar bloqueios temporários por excesso de requisições (*rate limiting*), o que deixaria as telas em branco.

Mitigação: Implementar tratamento global de erros via *Axios Interceptors* para exibir mensagens amigáveis (ex: "Servidores da FIA indisponíveis"). Além disso, manter dados de *fallback* (arquivos JSON locais com dados de corridas passadas) para garantir a continuidade do desenvolvimento em caso de queda da API.

- **Risco 2: Performance na Tela de Telemetria (Real-Time)**

Problema: Consumir e renderizar dados em tempo real (*Live Timing*) pode disparar múltiplas atualizações de estado por segundo, causando gargalos de renderização e travamentos no navegador do usuário.

Mitigação: Utilizar o *Zustand* com seletores granulares, garantindo que apenas os componentes que realmente mudaram sejam re-renderizados. Empregar *hooks* de otimização (*useMemo* e *useCallback*) na tabela de tempos e controlar a frequência das requisições (ex: *polling* a cada 10 segundos no máximo).

- **Risco 3: Esgotamento do Plano Gratuito do Firebase**

Problema: Erros de lógica no código (como *loops* infinitos no React Router ao verificar se o usuário está logado) podem disparar milhares de chamadas ao Firebase Auth, esgotando o limite do plano gratuito e bloqueando os acessos.

Mitigação: Implementar a persistência local da sessão de forma correta e testar exaustivamente o ciclo de vida dos componentes (*useEffect*) que fazem a validação de rotas protegidas antes de enviar para produção.

- **Risco 4: Fuga de Escopo (Scope Creep)**

Problema: A riqueza estatística da Fórmula 1 torna muito tentadora a adição de painéis, gráficos e regras extras, o que pode atrasar a entrega da Atividade 01.

Mitigação: Seguir de forma rigorosa e inegociável o *Sitemap* definido no Tópico 4. Nenhuma funcionalidade extra ou requisito *NICE* será iniciado antes que os seis requisitos obrigatórios (*MUST*) estejam completamente codificados e testados.

9. Cronograma Pessoal

Para garantir a entrega do escopo obrigatório com margem de segurança para testes e refatoração, o desenvolvimento foi dividido em quatro ciclos focados (*Sprints* semanais), priorizando a estabilidade estrutural antes da aplicação de detalhes visuais e lógicas complexas.

Semana 1: Infraestrutura e Roteamento (*Setup Sprint*)

Foco: Deixar a casa pronta e as portas abertas.

- Inicialização do projeto (*Vite + React + TypeScript*).
- Configuração de linter (*ESLint*) e formatador (*Prettier*).
- Instalação e configuração do *React Router* mapeando as rotas vazias do *Sitemap* (Tópico 4).
- Configuração inicial do *Firebase Authentication* (Criação do projeto no console e conexão com o *client*).

Semana 2: Consumo de Dados e Componentização Base (*Data Sprint*)

Foco: Trazer os dados reais da FIA para o *Front-End*.

- Configuração do *Axios* (criação da instância base e interceptadores de erro).
- Desenvolvimento dos serviços (*Services*) para consumo dos *endpoints* da *OpenF1* (Pilotos, Construtores e Calendário).
- Criação da *Store* do *Zustand* para gerenciamento do Tema Escuro (*Dark Mode*).
- Desenvolvimento dos componentes de *Layout* (Header fixo e Sidebar de navegação).

Semana 3: Interface e Funcionalidades Obrigatórias (*UI Sprint*)

Foco: Transformar os dados brutos nas telas mapeadas nos Wireframes.

- Desenvolvimento da *Home/Landing Page*.
- Implementação das Tabelas de Classificação com abas (*Tabs*) para Pilotos e Construtores (MUST-02).
- Criação da grade (*Grid*) do Calendário da Temporada (MUST-03).
- Implementação da página dinâmica de Perfil do Piloto (MUST-04).

Semana 4: Área Protegida, Refinamento e Entrega (*Polish Sprint*)

Foco: Ferramentas analíticas e polimento visual.

- Finalização do fluxo de Login integrado ao Firebase (MUST-01) e proteção das rotas (/app).
- Implementação do Comparativo *Head-to-Head* (MUST-05) e do painel de Telemetria ao Vivo (MUST-06).
- Estilização final via SCSS, garantindo a aplicação do *Dark Mode* e responsividade básica.
- Revisão final contra os critérios do edital (Checklist do Professor), *deploy* (opcional) e envio da atividade.